



UTEC

UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

Proyecto Final 1

Curso: Introducción a la Ciencia de la Computación

Profesor: Ian Paul Brossard

Periodo Académico: Pregrado 2025-2

Integrantes:

1. James Brad Macote Mattos
2. Treicy Rubí Calsina Gutierrez
3. Leonardo Alexander Landeo Huatay
4. Jared Eloy Daniel Chala Lastra
5. Julian Edison Alvarez Cartolin

Codigo:

202520158
202520007
202520151
202520040
202520140

Participación:

20%
20%
20%
20%
20%

Participación:

- James Brad Macote Mattos
 - Redacción de la pregunta 1 (pasos 1-4)
 - Código de la pregunta 1 (1º parte)
- Treicy Rubí Calsina Gutierrez
 - Redacción de Antecedentes
 - Redacción de la pregunta 2 (pasos 1 - 3)
 - Redacción de la pregunta 3 (pasos 1 - 4)
- Leonardo Alexander Landeo Huatay
 - Código de la pregunta 2
 - Código de la pregunta 3
 - Redacción de la pregunta 1 (investigar TF-IDF)
- Jared Eloy Daniel Chala Lastra
 - Portada
 - Redacción de los Fundamentos Teóricos
 - Redacción de la pregunta 1 (pasos 5 - 10)
- Julian Edison Alvarez Cartolin
 - Redacción de la pregunta 2 (pasos 4 - 6)
 - Código de la pregunta 1 (2º parte)
 - Redacción de la pregunta 3 (pasos 4 - 8)

1. Antecedentes

1.1. Contexto

Los Pokémon se caracterizan por su enorme diversidad, con más de 700 criaturas de distintas formas y tamaños. Existen un total de 18 tipos Pokémon, como fuego, agua, planta, psíquico, entre otros. Cada uno con sus propias fortalezas y debilidades. Este sistema es la base de la estrategia: por ejemplo, un Pokémon de tipo Fuego es fuerte contra uno de tipo Planta, pero débil frente a uno de tipo Agua. Esta dinámica hace que la composición de tu equipo, el posicionamiento y las tácticas que emplees sean decisivas para ganar los combates.

El agrupamiento de los Pokémon en función de sus movimientos y similitudes representa una herramienta valiosa para comprender mejor las relaciones existentes entre los distintos personajes de la franquicia. Mediante el uso de técnicas de Procesamiento de Lenguaje Natural (NLP), como TF-IDF y n-gramas, es posible transformar estas descripciones textuales en representaciones numéricas que permiten a la computadora identificar patrones y similitudes entre los Pokémon.

1.2. Descripción del dataset

El conjunto de datos utilizado proviene del sitio web de estrategia competitiva smogon.com, una plataforma ampliamente reconocida dentro de la comunidad de Pokémon por ofrecer información detallada sobre las características, habilidades y movimientos de los pokemons. El dataset proporcionado contiene un amplio registro en el cual tenemos a varios Pokémon distribuidos en cada fila, acompañado de diversas columnas que describen sus atributos principales. Entre ellas se incluye el nombre del Pokémon analizado, el enlace de origen (URL) desde donde se extrajo la información, una breve descripción textual que resume sus rasgos y habilidades, que nos ayudaran a poder clasificarlos con la ayuda del NLP.

2. Fundamento Teórico

2.1. TF-IDF (Term Frequency-Inverse Document Frequency):

TF-IDF es una técnica de procesamiento de lenguaje natural frecuentemente utilizada para representar documentos en forma vectorial para su posterior análisis (Manning, Raghavan, & Schütze, 2008). El TF mide cuantas veces aparece una palabra, y el IDF reduce la importancia de las palabras que se repiten mucho en todos los textos. En este proyecto, se usó TF-IDF para convertir los movimientos de los pokémon en valores numéricos. Así, el algoritmo pudo reconocer que tipos de movimientos son más representativos de cada pokemon y agruparlos según sus similitudes.

2.2. n-gramas:

Los n-gramas son una técnica de procesamiento de lenguaje natural que nos permite extraer secuencias contiguas de n elementos. Es comúnmente utilizada para analizar patrones en el texto y extraer características relevantes (Jurafsky & Martin, 2021; Manning, Raghavan & Schütze, 2008). Su principal función en el proyecto es identificar las combinaciones de palabras claves para analizar/clasificar los

ataques de los pokemones, movimientos, nombres de los tipos de cada ataque, entre otras características a considerar.

2.3. Clustering (K-Means):

El clustering o agrupamiento es una técnica de aprendizaje automático no supervisado, cuyo propósito principal es el de identificar patrones en un conjunto de datos y organizar los elementos dentro de clusters en base a sus similitudes. Esta técnica es especialmente útil para la minería de datos, pues no requiere de etiquetas predefinidas (Bishop, 2006; Han, Kamber & Pei, 2012). En Python, la clase KMeans del submódulo `sklearn.cluster` permite implementar esta técnica mediante la minimización de la distancia euclidiana a los centroides (Scikit-learn, 2025).

3. Métodos y desarrollo

- **PREGUNTA 1: Agrupamiento mediante TF-IDF**

El **TfidfVectorizer** es una clase de la biblioteca *scikit-learn* de Python, representada por un algoritmo matemático que permite convertir cadenas de texto en representaciones numéricas.

Funcionamiento general

Este algoritmo se compone de dos etapas fundamentales para representarlos de mejor manera usaremos un ejemplo con los siguientes datos:

ID	Texto
0	"A los científicos de la computacion les encanta la matematica"
1	"A los abogados les encanta leer libros"
2	"A los biologos les apasiona la genetica y la quimica"
3	"A los economistas les interesa la estadistica y la economia"

Paso 1:

En este paso comenzaremos formando **los n-gramas** de cada oración. Un **n-grama** básicamente son las combinaciones de palabras que podamos hacer en las oraciones. Por ejemplo:

1-Grama

En 1-Grama se organiza en una lista donde en los elementos solo hay una palabra. Ejemplo:

["a", "los", "científicos", "de", "la", "computacion", "les", "encanta", "la", "matematica"]

Ojo: Como puede observar por cada elemento hay solo 1 palabra

2-Grama

En un 2-Grama se organizará una lista donde hayan 2 palabras por elemento.

["a los", "los científicos", "científicos de", "de la", "la computacion", "computacion les", "les encanta", "encanta la", "la matematica"]

Ojo: Como puede observar por cada elemento en la lista hay 2 palabras

Paso 2:

Este paso se llama “Frecuencia de Término” o en inglés “TermFrequency”. Como su nombre indica, lo que intentamos hacer es determinar la frecuencia de aparición de cada n-grama en una oración.

Por ejemplo:

Si utilizamos 1-grama, tendríamos una lista como esta:

["a", "los", "científicos", "de", "la", "computacion", "les", "encanta", "la", "matematica"]

Analizando la aparición de cada término, obtendríamos algo así:

1-Grama	Frecuencia de aparición	Term-Frequency
a	1	0.1
los	1	0.1
científicos	1	0.1
de	1	0.1
la	2	0.2
computacion	1	0.1
les	1	0.1
encanta	1	0.1
matematica	1	0.1

Ojo: La fórmula para calcular el Term-Frequency es muy sencilla, se mostrara a continuación:

$$TF = \frac{\text{Número de veces que aparece la palabra en el documento}}{\text{Total de palabras en el documento}}$$

Paso 3:

En este paso utilizaremos un algoritmo de Frecuencia Inversa de Documento (IDF), el objetivo es demostrar la importancia de cada 1-grama en el documento asignándole un peso. Esto permite reducir la relevancia de palabras muy comunes que se repiten en muchos documentos, como “el”, “la”, “del”, etc. Usaremos esta fórmula:

$$IDF = \log \frac{\text{Total de documentos}}{1 + \text{Número de documentos donde aparece la palabra}}$$

Por ejemplo:

Palabra	Frecuencia Inversa de Documentos
a	-0.223
los	-0.223
cientificos	0.693
de	0.693
la	0.288
computacion	0.693
les	-0.223
encanta	0.288
matematica	0.693

Ojo: En Procesamiento de Lenguaje Natural a las oraciones se les llama Documentos.

Paso 4:

Como último paso, TfidfVectorizer multiplicará la frecuencia de cada término(**TF**) calculada anteriormente por la frecuencia inversa de documentos (**IDF**). De esta manera, cada palabra obtiene un peso que refleja su importancia tanto dentro del documento como en todo el corpus. A continuación, presentamos lo que retornaría la clase de sklearn como resultado para toda la base de datos anterior.

Palabra	Doc 0	Doc 1	Doc 2	Doc 3
a	-0.022	-0.028	-0.028	-0.028
los	-0.022	-0.028	-0.028	-0.028
cientificos	0.069	0	0	0
de	0.069	0	0	0
la	0.058	0.036	0	0
computacion	0.069	0	0	0
les	-0.022	-0.028	-0.028	-0.028
encanta	0.029	0.036	0	0
matematica	0.069	0	0	0
abogados	0	0.087	0	0
leer	0	0.087	0	0
libros	0	0.087	0	0
biologos	0	0	0.087	0
apasiona	0	0	0.087	0
genetica	0	0	0.087	0
y	0	0	0.036	0.036
quimica	0	0	0.087	0
economistas	0	0	0	0.087
interesa	0	0	0	0.087
estadistica	0	0	0	0.087
economia	0	0	0	0.087

Representación:

Los valores positivos representan palabras importantes y muy distintivas de cada documento.

Los valores negativos representan palabras que aparecen con frecuencia en todos los documentos y tienen poca importancia.

Los valores con 0 son palabras que no aparecen en cierto documento u oración.

Ahora mostraremos cómo aplicamos este conocimiento en el trabajo

Paso 5 Importación de librerías

Ahora que ya sabemos el funcionamiento del algoritmo TF-IDF, pasaremos a detallar la funcionalidad de nuestro código. En primer lugar, hemos importado las siguientes librerías/módulos:

- Clase TfidfVectorizer: Crearemos una instancia que nos permita aplicar el algoritmo TF-IDF a nuestra base de datos.
- Librería Pandas: Lo usaremos para manejar, manipular y organizar los datos,
- Clase KMeans: Crearemos una instancia que nos permita aplicar el algoritmo KMeans en la matriz.

```
##### Importacion #####  
from sklearn.feature_extraction.text import TfidfVectorizer  
import pandas as pd  
from sklearn.cluster import KMeans  
  
#####
```

El siguiente paso a realizar será leer el archivo CSV “smogon.csv” y convertirlo a una estructura de datos tipo Dataframe, todo esto mediante la función `pd.read_csv()`. Guardaremos este Dataframe en una variable llamada “Pokemon”.

```
##### Abrir CSV #####  
Pokemon = pd.read_csv("Project1/smogon.csv")
```

Luego copiaremos las columnas “moves” y “Pokemon” en las variables “MovimientosPokemon” y “NombresPokemon”, respectivamente.

```
MovimientosPokemon = Pokemon["moves"].copy()  
NombresPokemon = Pokemon["Pokemon"].copy()
```

Paso 6: Aplicar TF-IDF

Lo siguiente será crear un objeto de clase `TfidfVectorizer` con parámetro `gram_range=(1,1)`, que indica los unigramas, es decir, palabras individuales. Este objeto se guardará en la instancia “TFIDF”,

```
TFIDF = TfidfVectorizer(gram_range=(1,1))
MatrixNumericaTFIDF = TFIDF.fit_transform(MovimientosPokemon).toarray()
```

Para la siguiente línea de código, lo que se realizó fue lo siguiente:

- Primero, usamos el método `fit_transform()` del objeto vectorizador “TFIDF” para crear una matriz dispersa de la columna “MovimientosPokemon”.
- Segundo, convertimos la matriz dispersa en un arreglo de Numpy. Guardamos este arreglo en la variable “MatrixNumericaTFIDF” para posteriormente ser analizada con KMeans.

Paso 7: Aplicar KMeans

Ahora, aplicaremos KMeans sobre la matriz “MatrixNumericaTFIDF”. Para esto realizaremos lo siguiente:

- En primer lugar, crearemos un modelo de KMeans llamado “Cluster”, le asignaremos como parámetro `n_clusters=3` (Esto indica la cantidad de grupos/clusters en los que dividirá a los datos de “MatrixNumericaTFIDF”).
- A continuación, entrena el modelo con los datos mediante el método `.fit()`.
- Después de entrenar, el atributo `.labels_` devuelve el número de cluster asignado a cada elemento.

```
##### KMeans #####
Cluster = KMeans(n_clusters=3)
PokemonesAgrupados = Cluster.fit(MatrixNumericaTFIDF).labels_

#####
```

Paso 8: Obtener el número de columnas y tokens

Uno de los requerimientos de la **Pregunta 1** es mostrar el número de columnas y los tokens. Para cumplir con lo solicitado, se optó por lo siguiente:

- Número de columnas: Aplicación de la función `len()` sobre “MatrixNumericaTFIDF” y uso de `print()` para mostrar el número.
- Tokens: Usando el atributo `.vocabulary_` del vector TFIDF y `print()`, mostramos todos los tokens junto a su índice.


```
##### Numero de Columnas #####
NumeroColumnas = len(MatrixNumericaTFIDF[0])
print(f"Numero de columnas:\n{NumeroColumnas}")
#####

##### Tokens #####
Tokens = TFIDF.vocabulary_
print(f"Tokens:\n{Tokens}")
print("\n")

#####
```

Paso 9: Crear nuevo DataFrame

El siguiente paso será crear un DataFrame con la matriz y su vocabulario. Para lo cual, hicimos lo siguiente:

- Usamos el método `.get_feature_names_out()` para conseguir un array con los tokens de la matriz “MatrixNumericaTFIDF”.
- Convertimos la matriz numérica de TF-IDF (que hasta ahora era un arreglo numpy) en un DataFrame de pandas, que es mucho más legible.
- Lo imprimimos.

```
##### DataFrame con la matrix TFIDF #####
Vocabulario = TFIDF.get_feature_names_out()
TFIDF_DataFrame = pd.DataFrame(MatrixNumericaTFIDF, columns = Vocabulario)
print(f"Dataframe con la matrix TF-IDF y con el vocabulario:\n{TFIDF_DataFrame}")

#####
```

Paso 10: Crear el DataFrame final

La **Pregunta 1** nos solicita como evidencia principal un DataFrame que incluya la columnas del nombre del pokémon junto al cluster al que pertenece. Por tanto, hemos seguidos los siguientes pasos:

- En primer lugar, convertimos el array “PokemonesAgrupados” en una **serie de pandas** y etiqueta a la columna como “ClusterPokemones”
- Después, concatenamos la columna de “NombresPokemon” y “SeriesPokemonesAgrupados”

```
##### DataFrame final #####
SeriePokemonesAgrupados = pd.Series(PokemonesAgrupados, name = "ClusterPokemones")
PokemonDataSet = pd.concat([NombresPokemon, SeriePokemonesAgrupados], axis = 1)
```

Una vez conseguido Dataframe con los clusters, es necesario analizar el comportamiento de estos últimos para etiquetarlos mediante replace.

```
##### Interpretacion de clusters #####
PokemonDataSet["ClusterPokemones"] = PokemonDataSet["ClusterPokemones"].replace({
    0:"Movimientos Genericos",
    1:"Movimientos Especiales",
    2:"Movimiento Ofensivo",
})
```

Una vez concluido, con este último paso, convertiremos nuestro Dataframe en un archivo CSV, que podrá ser visualizado en la carpeta Project1.

```
##### Muestra y conversion a CSV #####
print(PokemonDataSet)
PokemonDataSet.to_csv("Project1/PokemonDataset(Ejercicio1).csv")
#####
```

- **PREGUNTA 2: Agrupamiento mediante un vocabulario controlado.**

Paso 1: Copia de la columna Pokemon

Se abre el archivo CSV que contiene la información de los Pokémon. Luego, se crea una copia de la columna “Pokemon” para guardarla aparte.

```
37 Pokemon = pd.read_csv("Project2/smogon.csv")
38 NombresPokemon = Pokemon['Pokemon'].copy()
```

Paso 2: Crear Vocabulario Controlado

En lugar de analizar todas las palabras del texto, se definió manualmente un **vocabulario controlado** compuesto por los 17 tipos de Pokémon y no se incluye el tipo “normal” :

```
41 ##### Vocabulario Controlado #####
42 VocabularioControlado = [
43     'bug', 'dark', 'dragon', 'electric', 'fairy',
44     'fighting', 'fire', 'flying', 'ghost', 'grass',
45     'ground', 'ice', 'poison', 'psychic', 'rock',
46     'steel', 'water'
47 ]
```

Esta lista de palabras funciona como filtro, de modo que el modelo TF-IDF solo considerará la frecuencia y relevancia de estos términos en la columna “moves”.

El propósito de este paso es evaluar si los Pokémon pueden agruparse adecuadamente usando únicamente los tipos de ataques que aprenden.

Paso 3: Limpiar de datos

Es necesario realizar una limpieza de datos de Pokemon[“moves”] para que las palabras del vocabulario sean detectadas correctamente.

- Usamos un bucle for que recorre palabra por palabra el VocabularioControlado.
- El contenido de la columna “moves” presenta palabras unidas sin espacios, lo que impide que el vectorizador las reconozca.
- Luego, se recorre cada tipo de Pokémon dentro del vocabulario y con la función replace() agrega espacios a la izquierda y derecha de las coincidencias encontradas.

Ejemplo: "fireblast" → " fire blast "

```
51 ##### Limpieza de datos #####
52 for TipoPokemon in VocabularioControlado:
53     Pokemon["moves"] = Pokemon["moves"].str.replace(TipoPokemon, f" {TipoPokemon} ", regex=False)
54 MovimientoPokemon = Pokemon["moves"].str.replace(r"\s+", " ", regex=True).str.strip()
```

Luego en Pokemon[“moves”] se aplica replace() para reemplazar espacios a uno solo y strip() para eliminar espacios al inicio y final del texto.

Este procedimiento garantiza que TfidfVectorizer pueda identificar las palabras de tipo correctamente como tokens independientes.

Paso 4: Implementando TfidfVectorizer

Una vez limpio el texto, se creo el objeto TFIDF y se utilizó la clase TfidfVectorizer con el parámetro vocabulary configurado según el VocabularioControlado, y el parámetro ngram_range=(1,1) para considerar únicamente unigramas (palabras individuales).

```
58 ##### TfidfVectorizer #####
59 TFIDF = TfidfVectorizer(vocabulary=VocabularioControlado,ngram_range=(1,1))
60 MatrixNumericaTFIDF = TFIDF.fit_transform(MovimientoPokemon).toarray()
```

Luego usamos fit_transform(): “Aprende” las palabras que hay en el texto y las convierte en números. En donde cada fila representa un Pokémon y cada columna una palabra del vocabulario. Finalmente, aplicamos .toarray() que convierte la matriz en un arreglo.

De este modo, cada Pokémon quedó representado mediante un vector numérico que refleja la frecuencia y relevancia de los tipos de movimientos que aparecen en su descripción. Esta matriz TF-IDF es base para realizar un análisis de similitud y agrupamiento posterior.

Paso 5: Empleando K-means

Para realizar el clustering se empleó el algoritmo K-Means, especificando 17 clusters, uno por cada tipo de Pokémon.

```
64 ##### KMeans #####
65 Clusters = KMeans(n_clusters=17,n_init=100,random_state=42)
66 PokemonesAgrupados = Clusters.fit(MatrixNumericaTFIDF).labels_
```

Se analizaron los valores TF-IDF de cada Pokémon y se agrupó según la similitud en la distribución de sus tipos de movimientos. De este modo, Pokémon con predominio de movimientos tipo fuego se agrupan juntos, al igual que los de tipo agua, psíquico, etc.

Paso 6: Resultado

Finalmente, convertimos la lista de clusters en una columna llamada "ClusterPokemon" con Series(). Luego, se creó un nuevo DataFrame combinando el nombre de cada Pokémon (NombresPokemon) con el número de cluster (PokemonesAgrupadosSerie) asignado mediante concat(). Este resultado se exportó a un archivo .csv llamado “PokemonDataSet(Ejercicio2).csv”.

```
70 ##### Base de datos (Resultado) #####
71 PokemonesAgrupadosSerie = pd.Series(PokemonesAgrupados,name="ClusterPokemon")
72
73 PokemonDataSet = pd.concat([NombresPokemon,PokemonesAgrupadosSerie],axis = 1)
74 PokemonDataSet.to_csv("Project2/PokemonDataSet(Ejercicio2).csv")
```

- **PREGUNTA 3: Verificación de agrupamiento por tipos**

Paso 1: Importación de librerías y carga del dataset

En primer lugar, se importaron las librerías necesarias:

```
29 ##### Importacion #####
30 from sklearn.feature_extraction.text import CountVectorizer
31 import pandas as pd
```

Luego, se cargó el archivo smogon.csv utilizando la función `pd.read_csv()`.

De este archivo se extrajo la columna “Pokemon” (que contiene los nombres de los personajes) que servirá como base para el análisis.

```
36 ##### Abrir CSV #####
37 Pokemon = pd.read_csv("Project3/smogon.csv")
38 NombresPokemon = Pokemon["Pokemon"].copy()
```

Paso 2: Definición del vocabulario controlado

Al igual que en la pregunta anterior, se definió manualmente un vocabulario controlado que incluye los 17 tipos elementales de Pokémon, los cuales serán las únicas palabras que se buscarán dentro de la columna “moves”.

```
42 ##### Vocabulario Controlado #####
43 VocabularioControlado = [
44     'bug', 'dark', 'dragon', 'electric', 'fairy',
45     'fighting', 'fire', 'flying', 'ghost', 'grass',
46     'ground', 'ice', 'poison', 'psychic', 'rock',
47     'steel', 'water'
48 ]
```

Este conjunto de palabras permitirá que el modelo cuente únicamente los movimientos relacionados con los tipos Pokémon y descarte cualquier otro término irrelevante.

Paso 3: Limpieza y preparación del texto

Antes de aplicar `CountVectorizer`, fue necesario preprocesar la columna “moves”.

El texto original contenía palabras unidas sin separación, lo cual impedía al modelo reconocerlas correctamente.

Para resolverlo, se implementó un ciclo for que recorre cada tipo dentro del vocabulario y agrega espacios a la izquierda y a la derecha de las coincidencias encontradas.

```
53 ##### Limpieza de datos #####
54 for TipoPokemon in VocabularioControlado:
55     Pokemon["moves"] = Pokemon["moves"].str.replace(TipoPokemon, f" {TipoPokemon} ", regex= False)
56 MovimientoPokemon = Pokemon["moves"].str.replace(r"\s+", " ", regex=True).str.strip()
```

Luego, se normaliza el texto reemplazando múltiples espacios por un único espacio y eliminando los espacios sobrantes al inicio o final.

Esto facilita que el CountVectorizer cuente correctamente las palabras de los tipos sean reconocidas de manera precisa por el vectorizador.

Paso 4: Aplicación de CountVectorizer

Una vez limpio el texto, se aplicó la clase CountVectorizer, indicando el vocabulario controlado como parámetro.

Este método convierte los textos de la columna “moves” en una matriz numérica en la que cada fila representa a un Pokémon y cada columna a uno de los 17 tipos.

El valor en cada celda indica cuántas veces aparece ese tipo en los movimientos del Pokémon correspondiente.

```
61 ##### CountVectorizer #####
62 Count = CountVectorizer(vocabulary=VocabularioControlado)
63 RepeticionesMatrixNumerica = Count.fit_transform(MovimientoPokemon).toarray()
```

De esta forma, se obtiene una representación cuantitativa del texto, lo que facilita analizar la frecuencia de aparición de cada tipo en los ataques de los Pokémon.

Paso 5: Creación del DataFrame

Con la matriz generada, se construyó un DataFrame con la función pd.DataFrame(), asignando como nombres de columnas los tipos Pokémon definidos en el vocabulario.

```
86 ##### Base de datos (Resultado) #####
87 #creacion:
88 PokemonDataSet = pd.DataFrame(RepeticionesMatrixNumerica, columns=VocabularioControlado)
```

Paso 6: Encontrar la habilidad dominante

Para encontrar la habilidad dominante se busca la columna con el valor más alto en cada fila, es decir, el tipo que más se repite en los movimientos del Pokémon.

Esto se logró mediante la función `idxmax(axis=1)`, la cual identifica la columna (tipo) con el valor más alto en cada fila.

De esta manera, cada Pokémon queda asociado a su tipo dominante según la frecuencia de los movimientos registrados en el dataset.

```
102 #Encontrar la habilidad dominante:
103 HabilidadMasFrecuente= PokemonDataSet[VocabularioControlado].idxmax(axis = 1)
104 print(f"Las palabras que mas se repiten (Tipo dominante) son:\n{HabilidadMasFrecuente}")
```

Paso 7: Agregar los resultados finales

Usamos `.insert()` para agregar una columna en la primera posición [0] con los nombres de Pokémon en `PokemonDataSet`. Y luego, agregamos otra columna con el tipo dominante.

Finalmente, se exportó la base de datos resultante a un archivo `.csv` llamado `Project3/PokemonDataSet(Ejercicio3).csv`

```
105 #Agregacion final de datos:|
106 PokemonDataSet.insert(0,"NombresPokemon",NombresPokemon)
107 PokemonDataSet.insert(1,"TipoDominante",HabilidadMasFrecuente)
108
109 print(PokemonDataSet)
110 PokemonDataSet.to_csv("Project3/PokemonDataSet(Ejercicio3).csv")
```

Paso 8: Combinar con datos de preguntas anteriores

Finalmente, se exportó la base de datos resultante a un archivo `.csv` llamado `PokemonDataSet(Ejercicio3).csv`, que contiene los siguientes elementos:

- El nombre del Pokémon
- Su tipo dominante
- Las frecuencias de aparición de cada tipo elemental

Este archivo servirá posteriormente para comparar el tipo dominante de cada Pokémon con los clusters obtenidos en las preguntas 1 y 2, permitiendo verificar si los grupos formados realmente reflejan la similitud entre los movimientos y los tipos de Pokémon.

```
109 ##### Agregacion de Habilidad dominante de Ejercicio 1 y 2 #####
110 #Agregacion Ejercicio 1:
111 PokemonDataSetEjercicio1 = pd.read_csv("Project1/PokemonDataSet(Ejercicio1).csv")
112 PokemonDataSetEjercicio1["TipoDominante"] = HabilidadMasFrecuente
113 PokemonDataSetEjercicio1.to_csv("Project3/DatosConHabilidadMasFrecuente/PokemonDataSetConHabilidadMasFrecuente(Ejercicio1).csv",index=False)
114 #Agregacion Ejercicio 2:
115 PokemonDataSetEjercicio2 = pd.read_csv("Project2/PokemonDataSet(Ejercicio2).csv")
116 PokemonDataSetEjercicio2["TipoDominante"] = HabilidadMasFrecuente
117 PokemonDataSetEjercicio2.to_csv("Project3/DatosConHabilidadMasFrecuente/PokemonDataSetConHabilidadMasFrecuente(Ejercicio2).csv",index= False)
```

4. Conclusiones

Julian:

Conclusión propia: En conclusión, el desarrollo de nuestro proyecto nos facilita comprender de forma más clara el procesamiento de lenguaje natural y así poder analizar datos textuales. Este trabajo en equipo me permitió obtener resultados más claros y cercanos a los tipos reales de cada Pokémon. De una manera sencilla, se demostró cómo el análisis de texto puede ayudarnos a descubrir patrones y relaciones entre los movimientos, permitiendo comprender mejor sus características y estilos de combate.

Conclusión del asistente virtual Copilot: Este proyecto permitió aplicar técnicas de procesamiento de lenguaje natural para analizar movimientos de Pokémon, utilizando TF-IDF para convertir texto en datos numéricos interpretables por algoritmos. Con K-Means, se agruparon Pokémon con características similares, revelando patrones relacionados con sus tipos elementales. El uso de un vocabulario controlado mejoró la precisión del análisis, enfocándose en términos clave. En conjunto, se reforzaron conocimientos de programación y análisis de datos, demostrando cómo la inteligencia artificial puede aplicarse en entornos creativos como los videojuegos, facilitando la clasificación automática de información y el descubrimiento de afinidades entre personajes de manera eficiente e innovadora.

Treicy:

Conclusión propia: En conclusión, este proyecto nos ayuda a comprender cómo funcionan distintas herramientas de aprendizaje automático (Machine Learning) y de Procesamiento de Lenguaje Natural (NLP) para analizar datos provenientes de la página Smogon. En este proyecto tomamos los movimientos de los Pokémon y aplicamos TfidfVectorizer junto con otros métodos como fit_transform para convertir textos en números. Además, usamos K-Means para agrupar Pokémon con movimientos y ataques similares. Finalmente, se empleó CountVectorizer para extraer la habilidad dominante de cada Pokémon.

Conclusión del asistente virtual (ChatGPT): En este proyecto se aplicaron conceptos fundamentales de Machine Learning y procesamiento de lenguaje natural para analizar datos de Pokémon obtenidos de Smogon. A través de herramientas como pandas y scikit-learn, se limpiaron y transformaron textos en representaciones numéricas mediante CountVectorizer y TfidfVectorizer. Esto permitió identificar patrones y determinar el tipo dominante de cada Pokémon según sus movimientos. Además, se integraron los resultados en diferentes etapas del proyecto, demostrando la importancia del preprocesamiento y la extracción de características. En conjunto, estos ejercicios fortalecen las habilidades prácticas en análisis de datos, manipulación de texto y aplicación de algoritmos, pilares esenciales en la formación profesional de un científico de la computación.

Jared:

Conclusión propia:

El presente proyecto de Machine Learning nos permitió comprender el funcionamiento de algoritmos como TF-IDF y K-Means, así como aplicar sus implementaciones de la librería scikit-learn para

analizar, organizar y manipular bases de datos como smogon.csv. Asimismo, fue posible identificar los casos en los que conviene emplear TF-IDF o CountVectorizer, según los requerimientos del análisis, así como las limitaciones de cada algoritmo. El conocimiento adquirido resulta de gran utilidad para futuros proyectos relacionados a los campos de la Minería de Datos o Procesamiento de Lenguaje Natural (NLP).

Conclusión del asistente virtual:

El proyecto logró aplicar de forma exitosa diversas técnicas de procesamiento de lenguaje natural (NLP) y aprendizaje no supervisado para analizar y clasificar información textual proveniente de los movimientos de los Pokémon. En el primer ejercicio, se utilizó TF-IDF junto con K-Means para realizar un agrupamiento inicial según el tipo de movimientos, identificando patrones generales, especiales y ofensivos. En el segundo ejercicio, se mejoró la precisión semántica mediante un vocabulario controlado, lo que permitió clasificar a los Pokémon en 17 grupos representativos de sus tipos elementales, fortaleciendo la coherencia interpretativa de los resultados. Finalmente, en el tercer ejercicio, se implementó CountVectorizer para cuantificar la frecuencia de aparición de los tipos dentro de los movimientos, determinando así la habilidad dominante de cada Pokémon.

En conjunto, el proyecto demuestra una comprensión sólida de las herramientas de scikit-learn, el manejo de datos con pandas, y el flujo completo de un proceso de minería de texto y clustering, evidenciando un enfoque analítico y bien estructurado para extraer conocimiento a partir de datos textuales.

Leonardo :

Conclusión Propia:

En conclusión, este proyecto nos permitió a mis compañeros y a mí comprender de una mejor manera cómo funcionan realmente herramientas como TF-IDF, n-gramas, CountVectorizer y K-Means dentro del procesamiento de lenguaje natural. Gracias a este proyecto, entendimos mejor cómo tecnologías como ChatGPT son capaces de interpretar y generar texto a partir de datos. Además, este proyecto de machine learning nos ayudó a poder entender cómo trabajar, limpiar y organizar grandes conjuntos de datos, lo cual fue fundamental para obtener resultados coherentes. Durante el proceso, observamos que los Pokémon con diferentes tipos de movimientos se agruparon en clústeres distintos, lo que demostró el correcto funcionamiento de nuestro programa y de los métodos implementados. En general, este proyecto fue muy interesante, ya que nos permitió aplicar los conocimientos adquiridos en clase, trabajar en equipo y aprender por nuestra cuenta nuevas herramientas que ampliaron nuestra comprensión del análisis de datos y del aprendizaje automático.

Conclusión con Chat-GPT:

El desarrollo de este proyecto permitió comprender de manera práctica cómo las técnicas de procesamiento de lenguaje natural (NLP) pueden aplicarse para analizar y agrupar información textual. A través del uso del método **TF-IDF (Term Frequency – Inverse Document Frequency)** y la implementación de **n-gramas**, fue posible representar los movimientos de los Pokémon como vectores numéricos que reflejan la importancia de cada término dentro del conjunto de datos. Este enfoque mostró que los personajes pueden agruparse de forma coherente en función de los tipos de ataques que aprenden, lo cual evidencia la capacidad de esta técnica para identificar patrones semánticos dentro de datos no estructurados.

Además, la aplicación del **vocabulario controlado** permitió restringir el análisis a los 17 tipos de Pokémon, demostrando que una selección adecuada de palabras clave puede mejorar la interpretabilidad de los resultados y facilitar la verificación de los agrupamientos. Este paso resultó esencial para evaluar la correspondencia entre los clusters generados y los tipos predominantes de cada personaje, brindando una validación concreta del proceso de clasificación.

Finalmente, este proyecto integró conocimientos de **programación en Python, manejo de datos con Pandas y scikit-learn, y fundamentos de machine learning no supervisado**. A nivel académico, permitió desarrollar pensamiento lógico, capacidad analítica y habilidades de experimentación computacional. En términos generales, se logró demostrar cómo el uso de técnicas computacionales permite transformar texto desordenado en información estructurada y significativa, cumpliendo así los objetivos propuestos del proyecto.

Brad:

Podemos concluir que gracias a las diferentes técnicas de TF-IDF, n-gramas y K-Means nos permitieron dar una gran ayuda al momento de poder analizar a los Pokemons según sus movimientos y tipos. Estas herramientas son una gran ayuda para organizar la base de datos proporcionada también mencionar que para este proyecto usamos como base la librería scikit-learn

Conclusión con Deepseek:

Este proyecto demostró la utilidad del Procesamiento de Lenguaje Natural (NLP) para agrupar Pokémon según sus movimientos. Mediante TF-IDF y K-Means, logramos identificar patrones y similitudes entre personajes, validando que los clusters reflejan tipos de movimientos predominantes. La implementación de vocabularios controlados y conteo de frecuencias permitió verificar la coherencia de los agrupamientos. La experiencia fortaleció nuestras habilidades en Python, pandas y scikit-learn, y consolidó nuestro trabajo colaborativo usando GitHub, aplicando con éxito técnicas de ciencia de datos a un contexto real y dinámico.

5. Reflexion final:

En este proyecto aprendimos a aplicar técnicas básicas de procesamiento de lenguaje natural y aprendizaje automático para analizar movimientos de Pokémon. Usando herramientas como TF-IDF, CountVectorizer y K-Means, pudimos transformar los textos de los movimientos en datos numéricos y agrupar a los Pokémon según sus características. Durante el trabajo, comprendimos cómo seleccionar palabras clave y organizar los datos de manera adecuada mejora la claridad de los resultados y facilita la identificación de patrones entre los tipos de Pokémon. Además, trabajar en equipo nos permitió combinar conocimientos, explorar nuevas funciones de Python y entender mejor cómo funcionan las librerías de análisis de datos como pandas y scikit-learn. En general, este proyecto nos mostró de manera práctica cómo aplicar conceptos de programación y análisis de datos para resolver problemas reales, reforzando nuestra capacidad de observación, organización y experimentación con información textual.

6. Bibliografía:

- 6.1. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- 6.2. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann Publishers.
- 6.3. Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing* (3rd ed., draft). Prentice Hall.
[Disponible en línea: <https://web.stanford.edu/~jurafsky/slp3/>]
- 6.4. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
[<https://nlp.stanford.edu/IR-book/>]
- 6.5. Scikit-learn. (2025). *sklearn.cluster.KMeans* — *scikit-learn documentation*. Recuperado de <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>